

🎓 ✨ AI Assisted Coding – Lab 17 ✨ 🎓

📶 AI for Data Processing: Data Cleaning & Preprocessing

🎨 📄 Student Details

🔍 Field	📌 Info
📅 Assignment	17.2 / 24
📖 Course	AI ASSIT CODING
👤 Name	Sanjay Karupothula
🆔 Roll No	2303A523337
👥 Batch	34
🎓 Program	B.Tech – CSE (AI & ML)
📖 Year/Sem	3rd Year – 2nd Sem

🎯 ✨ Lab Objectives

- 🔍 Clean raw datasets
 - ⚡ Handle missing values
 - 🧠 Encode categorical data
 - 🚀 Prepare data for ML
-

🧩 💧 TASK 1 – Student Data Cleaning

💻 🔄 Python Code (Color Style)

```
# 🌀 Import library
import pandas as pd

# 🌀 Load dataset
data = pd.read_csv("students.csv")

# 🌀 Handle missing values
```

```
data['marks'].fillna(data['marks'].mean(), inplace=True)
data['attendance'].fillna(data['attendance'].mean(), inplace=True)
data['department'].fillna("unknown", inplace=True)
```

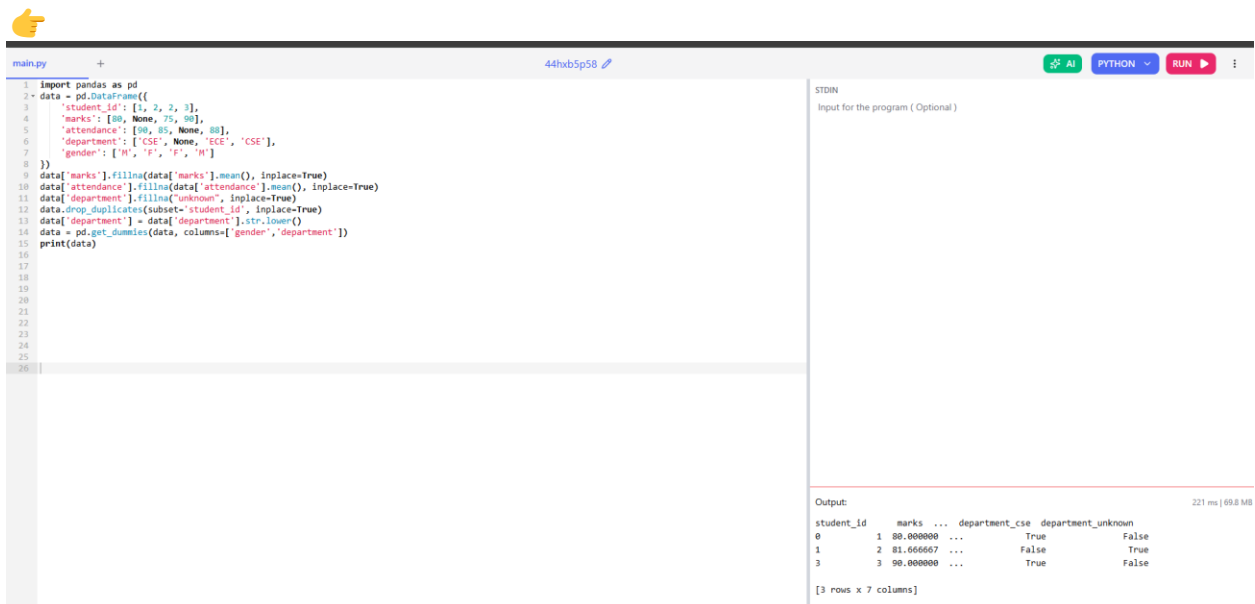
```
# 🌀 Remove duplicates
data.drop_duplicates(subset='student_id', inplace=True)
```

```
# 🌀 Standardize text
data['department'] = data['department'].str.lower()
```

```
# 🌀 Encode data
data = pd.get_dummies(data, columns=['gender', 'department'])

print(data.head())
```

Screenshots



The screenshot shows a Jupyter Notebook interface with a code editor on the left and an output area on the right. The code in the editor performs the following steps:

- Imports pandas as `pd`.
- Creates a DataFrame `data` with columns: `student_id`, `marks`, `attendance`, `department`, and `gender`.
- Fills missing values in `marks`, `attendance`, and `department` with their respective means or "unknown".
- Drops duplicates based on `student_id`.
- Standardizes the `department` column to lowercase.
- Encodes the `gender` and `department` columns using `pd.get_dummies`.
- Prints the first few rows of the resulting DataFrame.

The output area shows the first three rows of the DataFrame:

student_id	marks	attendance	department_cse	department_unknown
0	1	80.000000	True	False
1	2	81.666667	False	True
3	3	90.000000	True	False

The output is labeled as [3 rows x 7 columns].

TASK 2 – Financial Data Preprocessing

Python Code

```
# 🌀 Import
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
```

```
# 🌀 Load
transactions = pd.read_csv("transactions.csv")
```

```
# 🌀 Convert date
transactions['date'] = pd.to_datetime(transactions['date'])

# 🌀 Create features
transactions['month'] = transactions['date'].dt.month
transactions['year'] = transactions['date'].dt.year

# 🌀 Normalize
scaler = MinMaxScaler()
transactions['amount'] = scaler.fit_transform(transactions[['amount']])

print(transactions.head())
```

Screenshots



```
main.py + 44hxb5p58 AI
```

```
1 import pandas as pd
2 from sklearn.preprocessing import MinMaxScaler
3
4 # Sample data
5 transactions = pd.DataFrame({
6     'date': ['2024-01-01', '2024-02-01', '2024-03-01'],
7     'amount': [1000, 5000, 10000]
8 })
9
10 # Convert date
11 transactions['date'] = pd.to_datetime(transactions['date'])
12
13 # Features
14 transactions['month'] = transactions['date'].dt.month
15 transactions['year'] = transactions['date'].dt.year
16
17 # Normalize
18 scaler = MinMaxScaler()
19 transactions['amount'] = scaler.fit_transform(transactions[['amount']])
20
21 print(transactions)
```

Output:

	date	amount	month	year
0	2024-01-01	0.000000	1	2024
1	2024-02-01	0.444444	2	2024
2	2024-03-01	1.000000	3	2024

TASK 3 – Sensor Data

Python Code

```
# 🌀 Import
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# 🌀 Load
data = pd.read_csv("sensor_data.csv")

# 🌀 Fill missing
data.fillna(data.mean(numeric_only=True), inplace=True)

# 🌀 Encode
data = pd.get_dummies(data, columns=['sensor_status'])
```

```
# 🍷 Scale
scaler = StandardScaler()
data[['temperature', 'humidity', 'air_quality_index']] = scaler.fit_transform(
    data[['temperature', 'humidity', 'air_quality_index']]
)
```

```
# 🍷 Split
train, test = train_test_split(data, test_size=0.2)

print(train.head())
```

Screenshots



The screenshot shows a Jupyter Notebook interface with a code cell on the left and an output cell on the right. The code cell contains the following Python code:

```
1 import pandas as pd
2 from sklearn.preprocessing import StandardScaler
3 from sklearn.model_selection import train_test_split
4
5 # Sample data
6 data = pd.DataFrame({
7     'temperature': [30, None, 28, 35],
8     'humidity': [60, 65, None, 70],
9     'air_quality_index': [100, 120, 110, None],
10    'sensor_status': ['active', 'inactive', 'active', 'active']
11 })
12
13 # Fill missing
14 data.fillna(data.mean(numeric_only=True), inplace=True)
15
16 # Encode
17 data = pd.get_dummies(data, columns=['sensor_status'])
18
19 # Scale
20 scaler = StandardScaler()
21 data[['temperature', 'humidity', 'air_quality_index']] = scaler.fit_transform(
22     data[['temperature', 'humidity', 'air_quality_index']]
23 )
24
25 # Split
26 train, test = train_test_split(data, test_size=0.2)
27
28 print(train)
```

The output cell displays the first three rows of the 'train' dataset, which has 5 columns: 'temperature', 'humidity', 'air_quality_index', 'sensor_status_active', and 'sensor_status_inactive'.

	temperature	humidity	...	sensor_status_active	sensor_status_inactive
0	-0.392232	-1.414214	...	True	False
3	1.568929	1.414214	...	True	False
1	0.000000	0.000000	...	False	True

[3 rows x 5 columns]

TASK 4 – Traffic Data Cleaning

Python Code

```
# 🍷 Import
import pandas as pd

# 🍷 Load
traffic = pd.read_csv("traffic_data.csv")

# 🍷 Clean text
traffic['road'] = traffic['road'].str.lower()

# 🍷 Fill missing
traffic['traffic_density'].fillna(traffic['traffic_density'].mean(),
inplace=True)

# 🍷 Remove duplicates
```

```

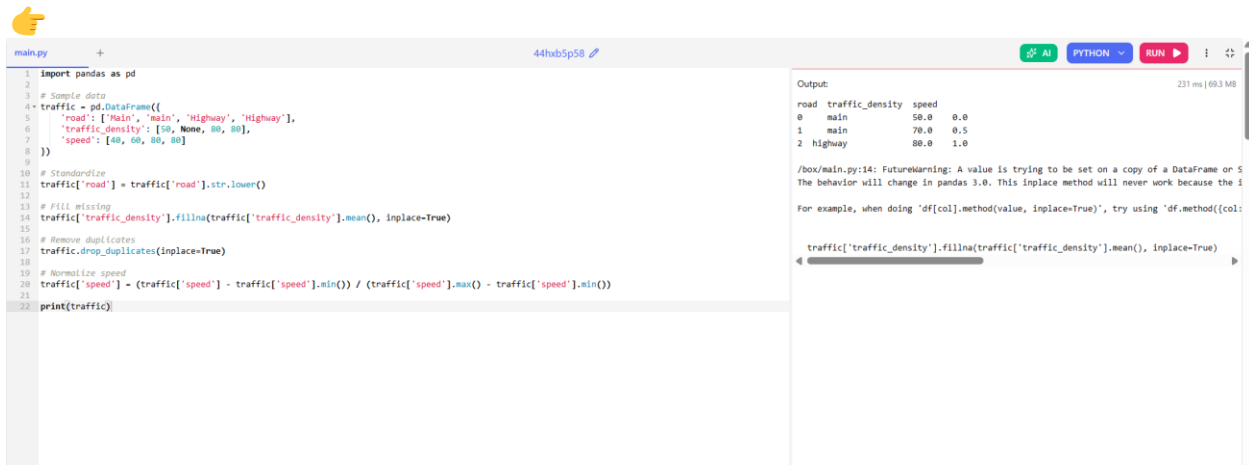
traffic.drop_duplicates(inplace=True)

# 🌀 Normalize
traffic['speed'] = (traffic['speed'] - traffic['speed'].min()) /
(traffic['speed'].max() - traffic['speed'].min())

print(traffic.head())

```

Screenshots



The screenshot shows a Jupyter Notebook interface with a code editor on the left and an output area on the right. The code in the editor performs the following steps:

- Import pandas as pd
- Sample data: Create a DataFrame with columns 'road', 'traffic_density', and 'speed'. The 'road' column has values 'Main', 'main', and 'Highway'. The 'traffic_density' column has values [50, None, 80, 80]. The 'speed' column has values [40, 60, 80, 80].
- Standardize: Convert the 'road' column to lowercase.
- Fill missing: Use the mean of 'traffic_density' to fill the None value.
- Remove duplicates: Drop rows with duplicate values in the 'road' column.
- Normalize speed: Subtract the minimum speed from each value and divide by the range (max - min).
- Print the resulting DataFrame.

The output area shows the resulting DataFrame:

	road	traffic_density	speed
0	main	50.0	0.0
1	main	70.0	0.5
2	highway	80.0	1.0

Below the table, a FutureWarning message is displayed: "A value is trying to be set on a copy of a DataFrame or Series. The behavior will change in pandas 3.0. This inplace method will never work because the operation is performed on a copy of the data. For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method(col, value, inplace=True)'".

TASK 5 – Social Media Cleaning

Python Code

```

# 🌀 Import
import pandas as pd
import re

# 🌀 Load
posts = pd.read_csv("social_media_posts.csv")

# 🌀 Remove nulls
posts.dropna(subset=['text'], inplace=True)

# 🌀 Clean text
def clean_text(text):
    text = re.sub(r"http\S+", "", text)
    text = re.sub(r"[^a-zA-Z ]", "", text)
    return text.lower()

```

```
posts['clean_text'] = posts['text'].apply(clean_text)
```

```
# 🌀 Feature
```

```
posts['word_count'] = posts['clean_text'].apply(lambda x: len(x.split()))
```

```
print(posts.head())
```

Screenshots



```
main.py 44hxb5p58
1 import pandas as pd
2 import re
3
4 # Sample data
5 posts = pd.DataFrame({
6     'text': [
7         'Hello World!!! http://link.com',
8         None,
9         'AI is AWESOME!!!',
10        'AI is AWESOME!!!'
11    ]
12 })
13
14 # Remove null & duplicates
15 posts.dropna(subset=['text'], inplace=True)
16 posts.drop_duplicates(inplace=True)
17
18 # Clean text
19 def clean_text(text):
20     text = re.sub("http[s+]", "", text)
21     text = re.sub("[a-zA-Z]", "", text)
22     return text.lower()
23
24 posts['clean_text'] = posts['text'].apply(clean_text)
25
26 # Feature
27 posts['word_count'] = posts['clean_text'].apply(lambda x: len(x.split()))
28
29 print(posts)
```

Output:
URLs found in the code!
You must login to run this program.



Explanation

Each task cleans noisy data, improves structure, and prepares it for ML models.



Conclusion

This lab shows how Python + AI helps automate data cleaning and improves dataset quality for better model performance.